CPR E 4890: Computer Networking and Data Communications
Lab Experiment #6 – CloudLab Experiment: IPv4 Routing Basics
(Total Points: 100)

Objective

Using CloudLab to get familiar with static routing by manually updating the routing tables of three nodes.

Lab Expectations

Work through the lab and let the TA know if you have any questions. After the lab, write up a lab report. Be sure to:

- Attend the lab. **(5 points)**
- Summarize what you learned in a few paragraphs. **(15 points)**
- Include your answers to all questions, with screenshots for *each* question. **(70 points)**
- Cleanup your resources, including a screenshot of successful cleanup. **(10 points)**

Procedure

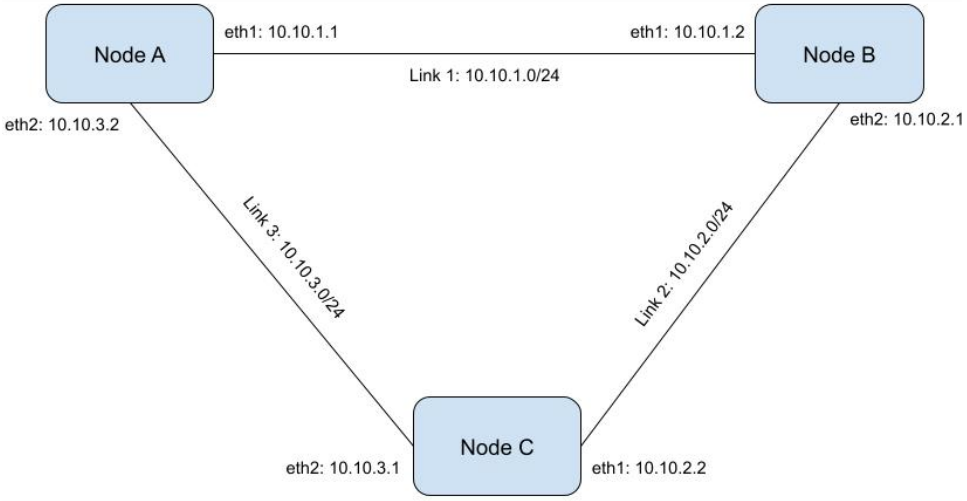
Prepare Your Experiment

1) If you haven't already, login to CloudLab at: https://www.cloudlab.us/login.php
2) Select **Experiments** and **Start Experiment**.
3) Select **Change profile**, search for **cpre4890s26-routing** and select it.
4) Click **Select Profile** and then **Next**.
5) Use your net-id for the name of the experiment and select an active cluster. Then select **Next** and then **Finish**.
6) Wait for your resources to be ready. This may take several minutes. If you receive an email regarding the image of your nodes being outdated, ignore it.
7) Log into the three nodes via SSH. Use the same technique you used in Lab 4 for using SSH.
8) On each node, install **`traceroute`** if it is not already available:

```
sudo apt install traceroute -y
```

Exercise 1: Examine the Network Topology

The network you will be working with is shown in the figure below.



This lab focuses on the **eth1** and **eth2** interfaces on each of the three nodes. The **eth0** interface will not be used, as it serves as the control interface for your SSH connection; modifying its routing may disconnect you from the node.

Note: Your interface names are typically **eth1** and **eth2**, but they may vary between experiment instances. Use **`ifconfig`** or **`ip addr show`** to confirm which interface is bound to which IP address on each node.

1) Begin by reading the ***entire*** man page for the route command (**`man route`** within your terminal).

2) Execute the **`route -n`** command in each of the three nodes to show their respective routing tables. The **"`-n`"** option prints the IP addresses rather than the assigned hostnames. All route screenshots should be done with the **"`-n`"** option. Include a screenshot of the tables in your lab report. **(10 points)**

3) From node A, try to **`ping`** the addresses of nodes B and C (two IP addresses for each node). Include a screenshot of the **`ping`** outputs and explain the results. **(10 points)**

4) What happens when you **`traceroute`** from A to IP address 10.10.2.2 before you set up the static routes? Why? Include a screenshot of the **`traceroute`** output in your lab report and explain the results. **(10 points)**

Exercise 2: Setting Up Static Routes

The following command will add the destination subnet **192.168.100.0/24** to the local routing table and use IP address **thisway** accessible via interface **intf** as the gateway:

Format:
```
sudo route add -net destiny netmask 255.255.255.0 gw thisway intf
```

Example:
```
sudo route add -net 192.168.100.0 netmask 255.255.255.0 gw 192.168.1.1 eth1
```

More specifically, in the above command:

- **`destiny`** refers to the subnet address (**A.B.C.0**) that indicates where packets should be sent when their destination IP address matches that subnet. For example, to direct traffic to the subnet **192.168.254.0/24**, set the destination to **192.168.254.0** with a netmask of **255.255.255.0** to reflex the **/24** prefix. (Note: A netmask of **255.255.255.0** should be sufficient for this lab.)
- **`intf`** is the name of the local interface (e.g., **eth0**) through which the traffic will be sent.
- **`thisway`** is the IP address (**W.X.Y.Z**) of the gateway interface that will receive the traffic.

Note: The gateway IP address **thisway** must be an address that the interface **intf** can access *directly* at Layer 2 (i.e., it should not require any further routing to reach **destiny**). Additionally, it must not be the IP address assigned to the interface on the current node.

To delete this entry from the table, simply replace **add** with **del**:
```
sudo route del -net destiny netmask 255.255.255.0 gw thisway intf
```

1) Modify the routing table so that node A can reach the IP addresses that were unreachable in **Exercise 1**. Include a screenshot of node A's updated routing table. **(10 points)**

2) Take a screenshot showing node A successfully performing both a **ping** and a **traceroute** to the IP address **10.10.2.2**. **(10 points)**

3) Configure additional static route(s) so that every node can reach all interfaces in the network. Take screenshots of updated routing tables for nodes B and C. **(10 points)**

4) From node B, perform a **traceroute** to each of the four interfaces it does not own. Take a screenshot of the output for each **traceroute**. **(10 points)**

   For example:

```
user@nodeb: $ traceroute 10.10.1.1
[OUTPUT]
user@nodeb: $ traceroute 10.10.3.2
[OUTPUT]
user@nodeb: $ traceroute 10.10.3.1
[OUTPUT]
user@nodeb: $ traceroute 10.10.2.2
[OUTPUT]
```

Note: The **`route`** command is part of the older *net-tools* package. The modern replacement is **`ip route`** from *iproute2*. The equivalent commands are:

- **View routes:** `ip route show` (replacing `route -n`)
- **Add a route:** `sudo ip route add 192.168.100.0/24 via 192.168.1.1 dev eth1`
- **Delete a route:** `sudo ip route del 192.168.100.0/24 via 192.168.1.1 dev eth1`

Cleanup

After completing the experiment, release your resources so others can use them. Terminate all resources and take a screenshot of the **Manage Resources** panel showing that your experiment has no remaining resources. **(10 points)**

Tips

- Use **`ifconfig`** or **`ip addr show`** to determine which Ethernet interface (e.g., **eth0**) is bound to which IP address on each node.
- **`ping`** and **`traceroute`** use ICMP packets that are sent to a destination, processed, and returned. Remember that the **response** also needs to know how to reach its own destination.
- **`tcpdump`** is a useful tool for debugging packet flow.
- For more information about **`route`**, refer to the manual page: **`man route`**
- For the modern equivalent, see **`man ip-route`** or run **`ip route help`** on your node.